RAPPORT DE TEST D'INTRUSION — SÉCURITÉ APPLICATIVE WEB

Analyse de la robustesse du processus de paiement

Évaluation en boîte noire d'une plateforme e-commerce

ÉTUDIANT

(à compléter)

DATE

(à compléter)

MODULE

Cybersécurité — Sécurité des applications web

CLASSIFICATION

Usage pédagogique — environnement autorisé

CIBLE

Boutique e-commerce (TP)

MÉTHODE

Boîte noire, côté client

RÉSUMÉ EXÉCUTIF

Synthèse des conclusions

L'évaluation a porté sur le tunnel de commande d'une boutique en ligne, afin de déterminer s'il était possible d'**imposer un prix au serveur** ou de contourner le paiement. Les tests montrent que la cible **résiste aux attaques de manipulation de prix** (le serveur recalcule les montants), mais qu'elle présente une **faiblesse de validation du jeton anti-CSRF**.

Domaine testé	Résultat	Sévérité
Imposition du montant de réduction (`cot_gv`)	Rejeté — recalcul serveur	SÉCURISÉ
Validation du jeton `securityToken`	Jeton invalide accepté	ÉLEVÉE
Rotation du jeton (rejeu)	Jeton statique, rejouable	MOYENNE
Exposition du cookie de session	Non lisible par le JS	SÉCURISÉ

— Sommaire

1. Objectif et périmètre

2. Méthodologie

3. Observation de la requête de commande

4. Imposition de prix — analyse conceptuelle

5. Résultats des tests

6. Évaluation de la protection CSRF

7. Remédiation

8. Conclusion

9. Annexe — structure des manipulations

01 Objectif et périmètre

Déterminer s'il est possible de contourner ou d'altérer le paiement d'une boutique en ligne, en particulier d'**imposer un prix** que le serveur accepterait sans le recalculer. Les tests ont été réalisés exclusivement dans l'**environnement autorisé du TP**, côté client, sans accès aux fichiers du serveur.

02 Méthodologie

Analyse en **boîte noire** via les outils développeur de Chrome (onglets *Network* et *Console*). Démarche itérative :

- Passer une commande normale et capturer la requête de validation.
- Identifier les paramètres transmis dans la requête.
- Pour chaque paramètre sensible : formuler une hypothèse d'abus et la tester.
- Observer la réponse du serveur pour statuer (accepté / rejeté).

03 Observation de la requête de commande

La validation s'effectue via une requête **POST** (formulaire classique). Paramètres observés dans l'onglet *Payload* :

Tableau 1 — Champs transmis lors de la validation de commande

Champ	Rôle probable
`securityToken`	Jeton anti-falsification (anti-CSRF)
`dc_redeem_code` / `gv_redeem_code`	Code de réduction / bon cadeau
`cot_gv`	Montant du bon appliqué au panier
`payment`	Mode de paiement sélectionné
`mtp_cc_*`	Données de carte bancaire
`mtp_timing` / `mtp_resolution`	Signaux anti-fraude (fuseau horaire, résolution)

Observation : le prix total n'est pas transmis comme paramètre librement modifiable — le serveur semble maîtriser le montant.

04 Imposition de prix — analyse conceptuelle

L'attaque dite d'« imposition de prix » exploite un serveur qui *fait confiance* à une valeur fournie par le client au lieu de la recalculer :

```
// Serveur naïf  : accepte "price=0"       → commande gratuite (vulnérable)
// Serveur sûr   : recalcule depuis sa base → ignore la valeur client (sécurisé)
```

Quel type de requête permettrait de « forcer » le serveur ?

Une requête (généralement POST) transportant, en paramètre modifiable, une valeur que le serveur devrait calculer lui-même. Formes typiques :

Tableau 2 — Formes génériques de requêtes vulnérables

Forme	Exemple générique
Prix envoyé par le client	`POST /panier/ajouter  product_id=42&price=0.01`
Montant total imposé	`POST /checkout  amount=0`
Quantité négative	`quantity=-1` → total nul ou négatif
Champ caché modifié	prix stocké dans un `<input type="hidden">`

Point clé : la requête vulnérable est rarement le paiement final (souvent durci), mais plutôt l'**ajout / mise à jour du panier**, là où le prix de la ligne est fixé pour la première fois.

05 Résultats des tests

T-01 — Falsification du montant de réduction (`cot_gv`)

SÉCURISÉ

Hypothèse	Le serveur accepte-t-il le montant de bon envoyé par le client ?
Manipulation	`document.querySelector('[name="cot_gv"]').value = "9999"`
Résultat	Le total à payer **n'a pas changé** : le serveur ignore la valeur et recalcule la réduction.
Verdict	Vecteur protégé — aucune confiance accordée à la donnée client.

[Capture d'écran à insérer — console + total inchangé]

T-02 — Falsification du jeton `securityToken`

VULNÉRABLE · ÉLEVÉE

Hypothèse	Le jeton anti-falsification est-il réellement vérifié côté serveur ?
Manipulation	`document.querySelector('[name="securityToken"]').value = "0000"`
Résultat	Malgré ce jeton invalide, le serveur **accepte la requête et poursuit la commande** (test arrêté avant l'envoi final).
Verdict	Protection présente dans le formulaire mais **non appliquée** côté serveur.

[Capture d'écran à insérer — jeton « 0000 » + commande qui continue]

06 Évaluation de la protection CSRF

Vérification des quatre critères d'un jeton anti-CSRF robuste, sans validation d'aucune commande :

Tableau 3 — Grille d'évaluation du jeton anti-CSRF

Critère	Attendu	Observé	Verdict
Imprévisibilité	jeton aléatoire, forte entropie	32 caractères hexadécimaux	OK
Validation serveur	rejet si absent / invalide	jeton `0000` accepté	ÉCHEC
Rotation / usage unique	nouveau jeton par requête	identique après rechargement	FAIBLE
Secret non exposé au JS	cookie `HttpOnly`	seuls des cookies d'analytics lisibles	OK

Conclusion : la protection CSRF est **inopérante**. Le jeton est bien aléatoire, mais il n'est pas validé côté serveur et reste rejouable. Le cookie de session, lui, n'est pas exposé au JavaScript (bon point).

07 Remédiation

- **Prix & réductions :** continuer à recalculer tous les montants côté serveur *(déjà en place ✔)*.
- **Jeton `securityToken` :** le valider systématiquement côté serveur, le lier à la session, le rendre à usage unique, et rejeter toute requête au jeton absent ou invalide.
- **Bornes de valeurs :** refuser quantités négatives/nulles et bons supérieurs au total.
- **Workflow :** garantir qu'aucune confirmation de commande n'est possible sans paiement validé (chaque étape contrôle la précédente).

08 Conclusion

La cible est **bien protégée contre l'imposition de prix** : le montant et la réduction sont recalculés côté serveur, et l'attaque classique échoue. En revanche, une **faille réelle** a été identifiée — le jeton `securityToken` n'est pas validé côté serveur — exposant l'application à la falsification de requêtes (CSRF). La recommandation prioritaire est la **validation stricte de ce jeton**. Ce résultat illustre un principe central de la sécurité web : *ne jamais faire confiance au client, et appliquer réellement les protections affichées.*

09 Annexe — structure des manipulations

Les commandes ci-dessous sont des **modèles génériques** (espaces réservés `NOM_DU_CHAMP` / `VALEUR`), fournis à titre illustratif pour documenter la méthode employée.

```
// 1) RECONNAISSANCE — lire la valeur actuelle d'un champ
document.querySelector('[name="NOM_DU_CHAMP"]').value

// 2) MANIPULATION — remplacer la valeur avant l'envoi
document.querySelector('[name="NOM_DU_CHAMP"]').value = "VALEUR_DE_TEST"

// 3) INSPECTION — lister les cookies lisibles côté client
document.cookie
```

Tableau 4 — Lecture des modèles

Élément	Rôle
`querySelector('[name="…"]')`	sélectionne un champ par son attribut `name`
`.value` (lecture)	phase de reconnaissance
`.value = "…"`	phase de test (injection d'une valeur)

Logique commune à tous les tests : **lire → modifier → valider → observer la réponse du serveur.**